

ESHARA — ClickUp Project Management

Sign Language Recognition System (ASL + ArSL)

Generated: February 22, 2026

HOW TO USE THESE PROMPTS IN CLICKUP

Paste each section as a **Space** → **Folder** → **List** → **Task** hierarchy. Each “TASK” block below = one ClickUp task card. Copy the “Description” block into the task description field. Use the Status labels: DONE / IN PROGRESS / TO DO / BLOCKED

SPACE: ESHARA — SLR System

FOLDER 1: ML MODELS

LIST 1.1 — English (ASL) Letter Model

Fingerspelling recognition for A-Z + space + del + nothing using MediaPipe MLP

TASK: [DONE] **Collect & Prepare ASL Letter Dataset**

Status: DONE **Description:** - Collected ASL alphabet image dataset (26 letters + space + del + nothing = 29 classes) - Stored in: **Letters**

Datasets/Asl_Sign_Data/asl_alphabet_train/ - Processed 50-image subset: asl_alphabet_train_50/ - Extracted MediaPipe hand landmarks (21 pts $\times$ 3 coords = 63 features per frame) - Saved to: Letters/ASL Letter (English)/asl_mediapipe_keypoints_dataset.csv - Dataset shape: (N rows $\times$ 64 columns: 63 features + 1 label)

TASK: [DONE] Train ASL Letter MLP Model (MediaPipe)

Status: DONE **Description:** - Notebook: Letters/ASL Letter (English)/Mediapipe_Training.ipynb
- Architecture: MLP (Multi-Layer Perceptron) on 63-dim landmark features - Input shape: (1, 63) — single frame - Output: 29 classes (A–Z, space, del, nothing)
- Final model: Letters/ASL Letter (English)/asl_mediapipe_mlp_model.h5
- Training history: training_history_initial.csv, training_history_continued.csv

TASK: [DONE] Train ASL Letter MobileNetV2 Model (Image-Based)

Status: DONE **Description:** - Notebook: Letters/ASL Letter (English)/MobileNetV2_Training.ipynb
- Architecture: Transfer Learning — MobileNetV2 base + custom classifier head - Input shape: (224, 224, 3) — RGB image - Output: 29 classes - Checkpoints: best_model_initial.h5, best_model_finetuned.h5, best_model_initial_optimized.h5 - Final model: sign_language_model_MobileNetV2.h5 (live test version: sign_language_model_MobileNetV2_updated.h5) - Draft notebooks: MBV2_draft.ipynb, MBV2_draft - Copy.ipynb

TASK: [DONE] ASL Letter Live Test (Combined Architecture)

Status: DONE **Description:** - Notebook: Letters/ASL Letter (English)/Combined_Architecture.ipynb
- Demonstrates real-time webcam recognition using both MLP and MobileNetV2 - Validates model performance on live camera feed - Draft: mediapipe_draft.ipynb, Mediapipe_Draft/

LIST 1.2 — Arabic (ArSL) Letter Model

Arabic fingerspelling recognition for Arabic alphabet letters using MediaPipe MLP

TASK: [DONE] Collect & Prepare ArSL Letter Dataset

Status: DONE **Description:** - Dataset CSV: Letters/ArSL Letter (Arabic)/Arabic Sign Language Letters Dataset.csv - Processed variants: arabic_final_training_data.csv, arabic_fixed_final.csv, arabic_fixed_with_nothing.csv, arabic_ready_for_training.csv - Main dataset (external): Letters Datasets/Dataset (ArASL)/ArASL Database/+ ArSL_Data_Labels.csv - Final cleaned dataset: Letters/ArSL Letter (Arabic)/Final Notebooks/FINAL_CLEAN_DATASET.csv - Fix scripts used: fix my csv.py, fix el fix.py, merge dataset.py

TASK: [DONE] Train ArSL Letter MLP Model (MediaPipe)

Status: DONE **Description:** - Notebooks: Mediapipe_Training.ipynb, Mediapipe_Final_Arabic.ipynb, Mediapipe_Optimized_Training.ipynb, Mediapipe_Training_GPU_Optimized.ipynb - Architecture: MLP on 63-dim MediaPipe landmarks - Input shape: (1, 63) - Output: Arabic letter classes - Final model: Letters/ArSL Letter (Arabic)/arsl_mediapipe_mlp_model_best.h5 - Also in Final Notebooks: arsl_mediapipe_mlp_model_final.h5 - Training history: training_initial.csv, training_finetime.csv, training_history_initial.csv - Confidence threshold set higher than ASL: 0.85 (Arabic model is stronger)

TASK: [DONE] Train ArSL Letter MobileNetV2 Model (Image-Based)

Status: DONE **Description:** - Notebooks: Mobilenet-arabic.ipynb, Mobilenet-arabic-optimized.ipynb, MobileNetV2_Mediapipe_Style_Training.ipynb, MobileNetV2_Training.ipynb - GPU-optimized training notebook: Mediapipe_Training_GPU_Optimized.ipynb - CPU variant: CBN-CPU.ipynb - Models: mobilenet_arabic_best_initial.h5, mobilenet_arabic_best_finetime.h5 - Final: Letters/ArSL Letter (Arabic)/Final Notebooks/mobilenet_arabic_final.h5

TASK: [DONE] ArSL Letter Live Test

Status: DONE **Description:** - Notebook: Letters/ArSL Letter (Arabic)/Final Notebooks/Combined_Architecture_Arabic_GPU.ipynb - Tests real-time Arabic letter recognition from webcam - Fix & verification scripts used during development: fixer notebook.ipynb

LIST 1.3 — English (ASL) Word Model

Continuous sign word recognition using BiLSTM + TemporalAttention on 30-frame sequences

TASK: [DONE] Prepare ASL Word Dataset (WLASL)

Status: DONE **Description:** - Dataset source: WLASL (World Level American Sign Language) video dataset - Dataset stored in: Words dataset/Words Datasets/WLASL_videos/ - Metadata: Words dataset/WLASL_v0.3.json - Class lists: Words dataset/wlasl_class_list.txt - Subset JSONs: nslt_100.json, nslt_300.json, nslt_1000.json, nslt_2000.json - Analysis tools: Words dataset/analyze_dataset.py, dataset_analysis.txt - Prep guide: Words dataset/PREP.md - Pre-extracted sequences: Words/ASL Word (English)/asl_word_sequences.npz - Class mappings: Words/ASL Word (English)/asl_word_classes.csv (157 rows: class_index → word_id) - Shared bilingual vocab: Words/Shared/shared_word_vocabulary.csv (157 entries: word_id → english + arabic + category)

TASK: [DONE] Design & Implement TemporalAttention Custom Layer

Status: DONE **Description:** - File: Separated Pipelines/shared/models/temporal_attention.py - Custom Keras layer for attention over time dimension - Required to load the BiLSTM word model - Architecture: 1. Alignment scores: $e = \tanh(x @ W + b)$ 2. Normalize: $a = \text{softmax}(e, \text{axis}=1)$ 3. Context: $c = \text{sum}(x * a, \text{axis}=1) \rightarrow \text{shape}(\text{batch}, \text{features})$ - Registered as custom_object when loading model via `tf.keras.models.load_model()`

TASK: [DONE] Train ASL Word BiLSTM Model

Status: DONE **Description:** - Notebooks: Words/ASL Word (English)/ASL_Word_Training.ipynb, ASL_Word_Training_1.ipynb, ASL_Word_Training_Kaggle.ipynb - Transformer experiment: asl-words-v2-transformer.ipynb - Trial notebook: asl-words-trial-1.ipynb - Architecture: BiLSTM + TemporalAttention - Input shape: (30, 63) — 30-frame window × 63 MediaPipe features - Output: 157 word classes - Final models: asl_word_lstm_model_best.h5, asl_word_lstm_model_final.h5 - Word confidence threshold: 0.35

TASK: [DONE] ASL Word Live Test

Status: DONE **Description:** - Notebook: Words/ASL Word (English)/ASL_Word_Live_Test.ipynb
- Tests real-time word recognition from webcam with 30-frame sequence buffering

LIST 1.4 — Arabic (ArSL) Word Model

Arabic word sign recognition — model training partially done,
dataset acquisition needed

TASK: [DONE] ArSL Word Dataset Download & Research

Status: DONE **Description:** - Notebook: Words/ArSL Word (Arabic)/Dataset
Downloading Notebook.ipynb - References: Words/ArSL Word (Arabic)/#
Code Citations.md - Target dataset: KArSL-502 (502 Arabic sign words) -
Kaggle training notebooks prepared: ArSL_Word_Training_Kaggle.ipynb

TASK: [IN PROGRESS] Train ArSL Word BiLSTM Model

Status: IN PROGRESS **Description:** - Notebooks: Words/ArSL Word
(Arabic)/ArSL_Word_Training.ipynb, ArSL_Word_Training_Kaggle.ipynb
- Live test notebook ready: Words/ArSL Word (Arabic)/ArSL_Word_Live_Test.ipynb
- Architecture planned: BiLSTM + TemporalAttention (same as ASL word
model) - Input shape: (30, 63) - Target model path: Words/ArSL Word
(Arabic)/arssl_word_lstm_model_best.h5 - **BLOCKER:** Need full KArSL-
502 dataset access to complete training - Word confidence threshold planned:
0.35

FOLDER 2: SHARED INFRASTRUCTURE (Separated Pipelines)

LIST 2.1 — Configuration

Central settings and configuration for all pipelines

TASK: [DONE] Create Central Config (settings.py)

Status: DONE **Description:** - File: Separated Pipelines/config/settings.py
- All model file paths (ASL/ArSL letters + words) - MediaPipe settings (complexity=0, detection_conf=0.7, tracking_conf=0.7, max_hands=2) - Inference settings: SEQUENCE_LENGTH=30, NUM_FEATURES=63 - English pipeline thresholds: letter_conf=0.80, word_conf=0.35, stable_window=5, majority_ratio=0.70, letter_cooldown=0.6s, word_cooldown=2.0s - Arabic pipeline thresholds: letter_conf=0.85, word_conf=0.35, stable_window=5, majority_ratio=0.70, letter_cooldown=0.7s, word_cooldown=2.0s - Motion detection thresholds: letter=0.015, word=0.030, buffer=5 frames - LLM settings: provider=openai, model=gpt-4o-mini, confidence_gate=0.75, time-out=2000ms, cache=500 entries, temperature=0.2 - FastAPI: port=8000, CORS origins configurable via env vars

LIST 2.2 — Shared Models

Reusable model utilities shared by both English and Arabic pipelines

TASK: [DONE] Model Loader Utility

Status: DONE **Description:** - File: Separated Pipelines/shared/models/model_loader.py
- load_keras_model(path, custom_objects): loads .h5 with TemporalAttention, forces float32 policy - load_label_encoder_from_csv(csv_path, label_column): extracts sorted unique labels from CSV - load_word_classes(csv_path): loads model_class_index → word_id dict - load_shared_vocabulary(csv_path): loads word_id → {english, arabic, category} dict - Auto-detects column names with fallbacks

TASK: [DONE] TemporalAttention Layer (Shared)

Status: DONE **Description:** - File: Separated Pipelines/shared/models/temporal_attention.py
- Custom Keras layer (see ML Models section for full spec) - Must be imported and registered before loading any word model

LIST 2.3 — Shared Utilities

Utilities used by both English and Arabic pipelines

TASK: [DONE] MediaPipe Hand Extractor

Status: DONE **Description:** - File: Separated Pipelines/shared/utils/mediapipe_extractor.py
- Class `MediaPipeExtractor`: - `extract(rgb_frame)` → (63,) float32 array or None (single dominant hand) - `extract_both_hands(rgb_frame)` → list of (63,) arrays (0-2 hands) - `close()` / context manager support - Uses settings from `config/settings.py` - Thread-safety note: create one instance per thread/pipeline

TASK: [DONE] Motion-Based Mode Detector

Status: DONE **Description:** - File: Separated Pipelines/shared/utils/mode_detector.py
- Class `ModeDetector`: - Detects whether signer is fingerspelling (LETTER mode) or doing a word sign (WORD mode) - Strategy: track mean absolute landmark displacement per frame - Smooth over 5 frames to avoid jitter (hysteresis band) - below `letter_threshold` → LETTER, above `word_threshold` → WORD, in between → keep previous - `SignMode` enum: LETTER / WORD / IDLE - Used by both English and Arabic pipelines identically

LIST 2.4 — LLM Correction Agent

GPT-powered confidence-gated correction layer for low-confidence predictions

TASK: [DONE] LLM Correction Agent

Status: DONE **Description:** - File: Separated Pipelines/llm_agent/correction_agent.py
- Called ONLY when model softmax confidence < `LLM_CONFIDENCE_GATE` (0.75 default) - Supports 4 correction modes: 1. English letter stream correction (spelling fix) 2. English word candidate reranking (context-aware) 3. Arabic letter stream correction () 4. Arabic word candidate reranking - LRU cache (500 entries) to avoid duplicate API calls - Returns `CorrectionResult` with: `corrected_text`, `was_corrected`, `used_llm`, `latency_ms`, `cached` - Async support with `LLM_TIMEOUT_MS=2000` fallback to raw prediction - Provider: OpenAI (gpt-4o-mini) — \$0.15/1M tokens - Data classes: `LetterCorrectionRequest`, `WordCorrectionRequest`, `CorrectionResult`

TASK: [DONE] LLM System Prompts

Status: DONE **Description:** - File: Separated Pipelines/llm_agent/prompts.py
- ENGLISH_LETTER_CORRECTION_PROMPT: ASL fingerspelling fixer, lists known misclassifications (M N, U V, A S T, G Q, D F, I J, R U) - ENGLISH_WORD_CORRECTION_PROMPT: context-aware word selection from candidates - ARABIC_LETTER_CORRECTION_PROMPT: MSA Arabic spelling fixer, lists known ArSL misclassifications (,) , etc.) - ARABIC_WORD_CORRECTION_PROMPT: Arabic context-aware word selection

LIST 2.5 — English Pipeline (Separated)

English (ASL) complete inference pipeline — letter + word combined

TASK: [TO DO] English Pipeline — Model Wrappers

Status: TO DO **Description:** - Folder: Separated Pipelines/english_pipeline/models/
- Create `__init__.py` is done; implement: 1. `EnglishLetterPredictor`: wraps MLP model, input (1,63), output label + confidence 2. `EnglishWordPredictor`: wraps BiLSTM model, manages 30-frame buffer, outputs top-k (word, confidence) pairs 3. `EnglishMobileNetPredictor`: wraps MobileNetV2, input (224,224,3), optional fallback - Use `model_loader.py` for loading, use thresholds from `settings.py`

TASK: [TO DO] English Pipeline — Inference Coordinator

Status: TO DO **Description:** - Folder: Separated Pipelines/english_pipeline/inference/
- Create `EnglishPipelineCoordinator` that: 1. Accepts each camera frame 2. Runs `MediaPipeExtractor.extract()` 3. Feeds to `ModeDetector.update()` → gets LETTER/WORD/IDLE 4. Routes to `EnglishLetterPredictor` or `EnglishWordPredictor` 5. Applies `LLMCorrectionAgent` when confidence < gate 6. Returns unified prediction result - Manages cooldown timers (letter_cooldown=0.6s, word_cooldown=2.0s)

TASK: [TO DO] English Pipeline — Letter Stream Decoder

Status: TO DO **Description:** - Folder: Separated Pipelines/english_pipeline/decoders/
- Port/integrate `letter_stream_decoder.py` from original project - Implements: stable window (5 frames), majority voting (70%), cooldown (0.6s) - Word decoder: maps BiLSTM output `word_ids` → vocabulary text, builds rolling sentence

LIST 2.6 — Arabic Pipeline (Separated)

Arabic (ArSL) complete inference pipeline — letter + word combined

TASK: [TO DO] Arabic Pipeline — Model Wrappers

Status: TO DO **Description:** - Folder: Separated Pipelines/arabic_pipeline/models/ (currently empty) - Create: 1. `ArabicLetterPredictor`: wraps ArSL MLP model, input (1,63), output Arabic letter + confidence 2. `ArabicWordPredictor`: wraps ArSL BiLSTM model (when trained), manages 30-frame buffer 3. `ArabicMobileNetPredictor`: wraps Arabic MobileNetV2 as optional fallback - Use thresholds: `letter_conf=0.85`, `word_conf=0.35`

TASK: [TO DO] Arabic Pipeline — Inference Coordinator

Status: TO DO **Description:** - Folder: Separated Pipelines/arabic_pipeline/inference/ (currently empty) - Create `ArabicPipelineCoordinator`: 1. Same flow as English coordinator 2. Use Arabic-specific thresholds (`letter_cooldown=0.7s` vs `0.6s` for English) 3. Returns Arabic letter/word text with proper RTL handling 4. Sends to Arabic LLM correction when `confidence < gate`

TASK: [TO DO] Arabic Pipeline — Letter Stream Decoder

Status: TO DO **Description:** - Folder: Separated Pipelines/arabic_pipeline/decoders/ (currently empty) - Arabic letter stream decoder with same logic as English but: - Arabic character handling (RTL, diacritics) - Arabic reshaper integration (`arabic-reshaper` + `python-bidi`) - Arabic-specific majority voting with similar-shaped letter groups

LIST 2.7 — Backend App Routes & Schemas

FastAPI app structure inside Separated Pipelines

TASK: [TO DO] Backend App Routes (REST + WebSocket)

Status: TO DO **Description:** - Folder: Separated Pipelines/backend/app/routes/ (currently empty) - Create: 1. `predict.py` — POST `/predict/letter`, POST

/predict/word 2. `websocket.py` — WebSocket /ws/recognize (real-time streaming) 3. `health.py` — GET /health (server status) 4. `language.py` — GET /languages (supported languages)

TASK: [TO DO] Backend App Schemas (Pydantic)

Status: TO DO **Description:** - Folder: Separated Pipelines/backend/app/schemas/ (currently empty) - Create: 1. `prediction_request.py` — Input: landmarks array (63 floats), language, mode 2. `prediction_response.py` — Output: prediction str, confidence float, mode, language, timestamp 3. `websocket_message.py` — WebSocket frame schema

LIST 2.8 — Tests

Unit and integration tests for all pipeline components

TASK: [TO DO] Write Unit Tests

Status: TO DO **Description:** - Folder: Separated Pipelines/tests/ (currently empty) - Tests to write: 1. `test_settings.py` — verify config paths resolve correctly 2. `test_model_loader.py` — test loading .h5 models + label encoders 3. `test_mediapipe_extractor.py` — test feature extraction from sample images 4. `test_mode_detector.py` — test LETTER/WORD/IDLE switching logic 5. `test_llm_agent.py` — test LRU cache, mock OpenAI calls 6. `test_english_pipeline.py` — end-to-end English inference test 7. `test_arabic_pipeline.py` — end-to-end Arabic inference test

FOLDER 3: BACKEND API (FastAPI)

LIST 3.1 — Backend Setup

TASK: [TO DO] Backend Project Setup & Dependencies

Status: TO DO **Description:** - Folder: SLR Main/backend/ - Create `requirements.txt`: - `fastapi==0.104.1` - `uvicorn[standard]==0.24.0` - `tensorflow==2.10.0` - `mediapipe==0.10.8` - `numpy==1.23.5` - `pandas==2.0.3` - `scikit-learn==1.3.2` - `python-dotenv==1.0.0` - `websockets==12.0` - `pydantic==2.5.2` - `python-multipart==0.0.6` - Create virtual environment: `python -m venv venv` - Create `.env` file with: `OPENAI_API_KEY`, `LLM_PROVIDER`, `BACKEND_PORT`, `CORS_ORIGINS` - Create `.env.example` template

TASK: [TO DO] Copy Model Files to Backend

Status: TO DO **Description:** - Create `scripts/copy_models.py`: - Copy `asl_mediapipe_mlp_model.h5` → `backend/model_files/` - Copy `arsl_mediapipe_mlp_model_final.h5` → `backend/model_files/` - Copy `asl_word_lstm_model_best.h5` → `backend/model_files/` - Copy all label CSV files → `backend/model_files/` - Verify all files exist after copy - Run and validate: `python scripts/copy_models.py` - Expected result: `backend/model_files/` contains 7+ files

LIST 3.2 — Backend Core Files

TASK: [TO DO] Backend Config & Entry Point

Status: TO DO **Description:** - Create `backend/app/__init__.py` - Create `backend/app/main.py`: - FastAPI app with title “ESHARA SLR API” - CORS middleware with configured origins - Include all routers - Startup event: load all models - `/docs` Swagger UI at root - Create `backend/app/config.py`: - Load from `.env` using `dotenv` - Model paths pointing to `backend/model_files/` - All thresholds (same as `settings.py`)

TASK: [TO DO] Backend Model Loading Module

Status: TO DO **Description:** - Create `backend/app/models/loader.py`: - `ModelRegistry` class — singleton that holds all loaded models - Load on startup: ASL MLP, ArSL MLP, ASL BiLSTM, shared vocabulary - Register `TemporalAttention` custom layer before loading word model - `get_asl_letter_model()`, `get_arsl_letter_model()`, `get_asl_word_model()` - Logging of load times and model summaries

TASK: [TO DO] Backend Letter Predictor

Status: TO DO **Description:** - Create `backend/app/models/letter_predictor.py`:
- `LetterPredictor` class - `predict(landmarks: np.ndarray, language: str) → (label, confidence)` - Accepts (63,) feature array - Routes to ASL or ArSL model based on `language` param (“en” / “ar”) - Returns top prediction + confidence score

TASK: [TO DO] Backend Word Predictor

Status: TO DO **Description:** - Create `backend/app/models/word_predictor.py`:
- `WordPredictor` class - Maintains 30-frame deque buffer per session/language
- `add_frame(landmarks, language) → updates buffer` - `predict(language) → List[(word, confidence)]` — top-k predictions - Resolves `word_id` → English + Arabic text via shared vocabulary

TASK: [TO DO] Backend Letter Decoder Module

Status: TO DO **Description:** - Create `backend/app/models/letter_decoder.py`:
- Port `letter_stream_decoder.py` to backend package - Per-session state: stable window (5 frames), majority voting (70%), cooldown (0.6s EN / 0.7s AR)
- `update(label, confidence, language) → Optional[str]` — returns committed letter or None - Arabic output with `arabic-reshaper` + `python-bidi` for proper display

TASK: [TO DO] Backend Word Decoder Module

Status: TO DO **Description:** - Create `backend/app/models/word_decoder.py`:
- `WordDecoder` class - Accumulates committed words into a sentence - Sentence builder: word + space logic, sentence context tracking - `add_word(word: str) → str` — returns full sentence so far - `clear()` — reset sentence - `get_sentence() → str`

TASK: [TO DO] Backend Mode Detector Module

Status: TO DO **Description:** - Create `backend/app/models/mode_detector.py`:
- Wrap/expose `ModeDetector` from shared utilities for backend use - Per-session state (different sessions may be in different modes) - Returns JSON-serializable mode string: “letter” / “word” / “idle”

LIST 3.3 — Backend Routes (API Endpoints)

TASK: [TO DO] REST Prediction Endpoints

Status: TO DO **Description:** - Create backend/app/routes/predict.py:
- POST /predict/letter — single frame prediction - Request body: {landmarks: [63 floats], language: "en"|"ar"} - Response: {prediction: "A", confidence: 0.95, mode: "letter", timestamp: ...} - POST /predict/word — add frame to word buffer + get word prediction - Request body: {landmarks: [63 floats], language: "en"|"ar"} - Response: {candidates: [{"word": "hello", "confidence": 0.7}], sentence: "my name is", timestamp: ...} - GET /health — server + models status - GET /models/info — loaded models metadata

TASK: [TO DO] WebSocket Real-Time Streaming Endpoint

Status: TO DO **Description:** - Create backend/app/routes/websocket.py:
- WS /ws/recognize — bidirectional real-time recognition - Client sends: {landmarks: [63 floats], language: "en"|"ar"} per frame - Server sends back: {mode: "letter"|"word", prediction: str, confidence: float, sentence: str} - Per-connection session state: ModeDetector, LetterDecoder, WordDecoder, WordPredictor buffer - Handles disconnects gracefully, cleans up session state - Target latency: < 50ms per frame

TASK: [TO DO] Backend Pydantic Schemas

Status: TO DO **Description:** - Create backend/app/schemas/:
- PredictLetterRequest(BaseModel): landmarks (List[float] len=63), language (Literal["en", "ar"]) - PredictWordRequest(BaseModel): landmarks (List[float] len=63), language, session_id optional - PredictionResponse(BaseModel): prediction, confidence, mode, language, timestamp - WordCandidates(BaseModel): candidates list, sentence str, language - HealthResponse(BaseModel): status, models_loaded dict, uptime

LIST 3.4 — Backend Infrastructure

TASK: [TO DO] Backend Dockerfile

Status: TO DO **Description:** - Create backend/Dockerfile: - Base: python:3.9-slim - Copy requirements.txt + install dependencies - Copy model files + app code - Expose port 8000 - CMD: uvicorn app.main:app --host 0.0.0.0 --port 8000 - Create backend/.dockerignore - Test: docker build -t eshara-backend . + docker run -p 8000:8000 eshara-backend

TASK: [TO DO] Backend Railway Deployment Config

Status: TO DO **Description:** - Create backend/railway.toml: - Build: Docker - Healthcheck: GET /health - Port: 8000 - Environment variables: OPENAI_API_KEY, CORS_ORIGINS - Create backend/Procfile as fallback: web: uvicorn app.main:app --host 0.0.0.0 --port \$PORT

FOLDER 4: WEB FRONTEND (React + TypeScript)

LIST 4.1 — Web Setup

TASK: [TO DO] Web Frontend Project Initialization

Status: TO DO **Description:** - Framework: React + TypeScript + Vite - Folder: SLR Main/web/ - Run: npm create vite@latest web -- --template react-ts - Install dependencies: - @mediapipe/hands — hand landmark detection - @mediapipe/camera_utils — camera abstraction - @mediapipe/drawing_utils — draw landmarks on canvas - tailwindcss + @tailwindcss/forms — styling - react-router-dom — routing - i18next + react-i18next — bilingual (EN/AR) support - Create tailwind.config.js with RTL plugin for Arabic - Target: runs at localhost:5173

LIST 4.2 — Web Core Architecture

TASK: [TO DO] Web App Entry Point & Routing

Status: TO DO **Description:** - Create `web/src/App.tsx`: - React Router with routes: `/` (Home), `/recognize` (Recognize), `/about` - Language context provider (EN/AR state) - Theme provider - Create `web/src/main.tsx` entry - Create `web/src/types/index.ts`: - `SignMode: "letter"|"word"|"idle"` - `Language: "en"|"ar"` - `PredictionResult{prediction, confidence, mode, sentence}` - `Landmark{x, y, z}`

LIST 4.3 — Web Custom Hooks

TASK: [TO DO] MediaPipe Hands Hook

Status: TO DO **Description:** - Create `web/src/hooks/useMediaPipe.ts`: - Initialize `@mediapipe/hands` with `complexity=0` (lite) - Connect to video element ref - Process each frame → emit `landmarks: number[]` (63 floats) - Emit `handDetected: boolean` - Handle camera permissions - Performance: target 30fps on mid-range laptop

TASK: [TO DO] WebSocket Connection Hook

Status: TO DO **Description:** - Create `web/src/hooks/useWebSocket.ts`: - Connect to `ws://localhost:8000/ws/recognize` (configurable via env) - `sendFrame(landmarks: number[], language: Language)` — sends per-frame data - Receive and parse `PredictionResult` - Auto-reconnect on disconnect (exponential backoff) - Connection state: `connecting` / `connected` / `disconnected` / `error` - Queue frames if connection temporarily lost

LIST 4.4 — Web Components

TASK: [TO DO] Camera Component

Status: TO DO **Description:** - Create `web/src/components/Camera.tsx`: - `<video>` element + `<canvas>` overlay - Connects `useMediaPipe` hook - Streams landmarks to `useWebSocket` - Props: `language`, `onPrediction`, `onModeChange` - Handles permissions denied with user-friendly message

TASK: [TO DO] Hand Landmarks Overlay Component

Status: TO DO **Description:** - Create `web/src/components/HandOverlay.tsx`:
- Canvas overlay drawn over video element - Uses `@mediapipe/drawing_utils`
to draw: - Hand skeleton connections - Landmark dots (color-coded by
confidence) - Bounding box - Smooth animations between frames

TASK: [TO DO] Prediction Display Component

Status: TO DO **Description:** - Create `web/src/components/PredictionDisplay.tsx`:
- Shows current predicted letter or word - Large bold character display with
fade animation - Confidence bar (color: green >80%, yellow 60-80%, red <60%)
- Mode badge (LETTER / WORD / IDLE) - Bilingual: shows Arabic text
RTL when `language=ar`

TASK: [TO DO] Sentence Builder Component

Status: TO DO **Description:** - Create `web/src/components/SentenceBuilder.tsx`:
- Running sentence display (accumulated letters/words) - Clear button - Copy
to clipboard button - Text-to-speech output button - RTL text direction when
Arabic mode - LLM correction indicator (shows when an AI correction was
applied)

TASK: [TO DO] Mode Indicator Component

Status: TO DO **Description:** - Create `web/src/components/ModeIndicator.tsx`:
- Visual indicator: LETTER mode (hand still icon) / WORD mode (moving
hand icon) / IDLE (no hand icon) - Smooth transition animation between
modes - Shows current confidence threshold

TASK: [TO DO] Language Toggle Component

Status: TO DO **Description:** - Create `web/src/components/LanguageToggle.tsx`:
- EN AR toggle button - Switches all UI text + model predictions - Persists
preference to `localStorage` - Adjusts text direction (LTR RTL) for entire app

LIST 4.5 — Web Pages

TASK: [TO DO] Home Page

Status: TO DO **Description:** - Create `web/src/pages/Home.tsx`: - Hero section: project name ESHARA + tagline - Quick demo GIF or static image - “Start Recognizing” CTA button → `/recognize` - Language selector - Team info section

TASK: [TO DO] Recognition Page (Main Page)

Status: TO DO **Description:** - Create `web/src/pages/Recognize.tsx`: - Main recognition view layout: - Left: Camera feed with `HandOverlay` - Right: `PredictionDisplay` + `SentenceBuilder` - Top bar: `ModeIndicator` + `LanguageToggle` - Responsive layout (mobile: stacked) - Loading state while models initialize - Error state when camera denied

LIST 4.6 — Web Services & Utilities

TASK: [TO DO] API Client Service

Status: TO DO **Description:** - Create `web/src/services/api.ts`: - `BASE_URL = import.meta.env.VITE_API_URL || "http://localhost:8000"` - `predictLetter(landmarks, language) → PredictionResult` (REST fallback) - `getHealth() → HealthStatus` - `getModelsInfo() → ModelsInfo` - Axios or fetch with error handling + retry

TASK: [TO DO] Landmark Utilities

Status: TO DO **Description:** - Create `web/src/utills/landmarks.ts`: - `normalizeLandmarks(raw: MediaPipeLandmarks) → number[]` → 63-float array - `drawLandmarks(ctx, landmarks, options)` — canvas drawing util - `computeMotion(prev, curr) → number` — motion score for mode detection (client-side preview)

TASK: [TO DO] Web i18n (Bilingual Translations)

Status: TO DO **Description:** - Create `web/src/i18n/en.json` — English UI strings - Create `web/src/i18n/ar.json` — Arabic UI strings - Keys: nav items, button labels, status messages, error messages - RTL support: `dir="rtl"` on html element when Arabic

TASK: [TO DO] Web Environment Config

Status: TO DO **Description:** - Create `web/.env.local`: `VITE_API_URL=http://localhost:8000`
- Create `web/.env.production`: `VITE_API_URL=https://your-railway-app.railway.app`
- Create `web/vercel.json`: rewrites for SPA routing

FOLDER 5: MOBILE APP (React Native + Expo + TFLite)

LIST 5.1 — Mobile Setup & Model Conversion

TASK: [TO DO] Convert Models to TFLite for Mobile

Status: TO DO **Description:** - Create `scripts/convert_models.py`: - Convert `asl_mediapipe_mlp_model.h5` → `asl_letter.tflite` - Convert `arsl_mediapipe_mlp_model_final.h5` → `arsl_letter.tflite` - Convert `asl_word_lstm_model_best.h5` → `asl_word.tflite` (custom op for TemporalAttention) - Output label JSONs: `asl_letter_labels.json`, `arsl_letter_labels.json`, `asl_word_vocab.json` - Output destination: `mobile/assets/models/` - Validate: run each `.tflite` with test input, compare output to `.h5` output

TASK: [TO DO] Export Label Encoders to JSON

Status: TO DO **Description:** - Create `scripts/export_labels.py`: - Load `asl_mediapipe_keypoints_dataset.csv` → extract labels → save `asl_letter_labels.json` - Load `FINAL_CLEAN_DATASET.csv` → extract labels → save `arsl_letter_labels.json` - Load `asl_word_classes.csv` + `shared_word_vocabulary.csv` → save `asl_word_vocab.json` - Format: `{"labels": ["A","B",...], "language": "en"}`

TASK: [TO DO] Expo React Native Project Setup

Status: TO DO **Description:** - Folder: SLR Main/mobile/ - Run: `npx create-expo-app mobile --template` - Install dependencies: - `expo-camera` — camera access - `@mediapipe/hands` (or Expo MediaPipe plugin) — hand detection - `react-native-fast-tflite` — TFLite inference on-device - `expo-file-system` — model file access - `react-navigation` — navigation - `react-native-paper` — UI components - `i18next` + `react-i18next` — bilingual support - Test on device with Expo Go app

LIST 5.2 — Mobile Services

TASK: [TO DO] TFLite Model Service

Status: TO DO **Description:** - Create `mobile/src/services/tfliteModel.ts`:
- Load `.tflite` models from `assets/models/` - `LetterModel`: `predict(landmarks: Float32Array) → {label: string, confidence: number}` - `WordModel`: manages 30-frame buffer, `addFrame(landmarks)`, `predict() → WordCandidate[]`
- Lazy loading — models loaded once on first use - Error handling for model load failures

TASK: [TO DO] MediaPipe Mobile Service

Status: TO DO **Description:** - Create `mobile/src/services/mediapipe.ts`:
- Interface with camera frames from Expo Camera - Extract 63-dim hand landmarks per frame - Returns landmarks or null (no hand detected) - Performance target: 20fps on mid-range Android/iOS

TASK: [TO DO] Mobile Letter & Word Decoders

Status: TO DO **Description:** - Create `mobile/src/services/decoder.ts`:
- Port `letter_stream_decoder.py` logic to TypeScript: - Stable window (5 frames), majority voting (70%), cooldown (0.6s EN / 0.7s AR) - `LetterDecoder.update(label, confidence, language) → string | null`
- `WordDecoder.addWord(word, language) → string` — returns full sentence
- `SentenceBuilder.clear() / getSentence()`

TASK: [TO DO] Mobile Landmark Utilities

Status: TO DO **Description:** - Create `mobile/src/utils/landmarks.ts`:
- `normalizeLandmarks(raw) → Float32Array` — 63 floats for TFLite input
- `computeMotion(prev, curr) → number` — mode detection score
- `drawHandSkeleton(canvas, landmarks)` — for overlay rendering

LIST 5.3 — Mobile Screens

TASK: [TO DO] Home Screen

Status: TO DO **Description:** - Create `mobile/src/screens/HomeScreen.tsx`:
- ESHARA logo + app name - “Start” button → navigate to `RecognizeScreen` -
Language selector (EN/AR) - About/info section

TASK: [TO DO] Recognition Screen (Main)

Status: TO DO **Description:** - Create `mobile/src/screens/RecognizeScreen.tsx`:
- Full-screen camera view - Hand landmarks overlay - Bottom panel: current prediction + confidence + sentence - Mode indicator: letter/word/idle -
On-device inference loop: 1. Camera frame → `MediaPipe` → landmarks 2. `ModeDetector` → LETTER or WORD 3. TFLite inference → prediction 4. `Decoder` → committed text 5. `SentenceBuilder` → display

TASK: [TO DO] Settings Screen

Status: TO DO **Description:** - Create `mobile/src/screens/SettingsScreen.tsx`:
- Language toggle: English ASL / Arabic ArSL - Confidence threshold sliders -
Mode: auto-detect / force letter / force word - Clear sentence history - About app

LIST 5.4 — Mobile Components

TASK: [TO DO] Camera View Component (Mobile)

Status: TO DO **Description:** - Create `mobile/src/components/CameraView.tsx`:
- Expo Camera + permission handling - Frame callback at target FPS - Canvas overlay for landmarks

TASK: [TO DO] Result Overlay Component (Mobile)

Status: TO DO **Description:** - Create `mobile/src/components/ResultOverlay.tsx`:
- Floating overlay showing: - Current prediction letter/word (large, bold) -
Confidence percentage - Mode badge - Running sentence (scrollable)

TASK: [TO DO] Mobile App Entry Point & Navigation

Status: TO DO **Description:** - Create `mobile/App.tsx`: - React Navigation stack - Language context provider - Splash screen with ESHARA logo - Permission requests on first launch

FOLDER 6: CLOUD DEPLOYMENT

LIST 6.1 — Docker & Local Dev

TASK: [TO DO] Docker Compose (Local Dev Environment)

Status: TO DO **Description:** - Create `docker-compose.yml` at project root: -
backend service: build `./backend`, port 8000:8000 - web service (optional): build
`./web`, port 5173:5173 - Shared `model_files` volume - Environment variables
from `.env` file - Test: `docker-compose up --build`

LIST 6.2 — Backend Deployment (Railway)

TASK: [TO DO] Deploy Backend to Railway

Status: TO DO **Description:** - Platform: Railway.app (free tier for prototyping) - Steps: 1. Push backend code to GitHub 2. Connect repo to Railway 3. Set environment variables: `OPENAI_API_KEY`, `CORS_ORIGINS` 4. Railway auto-detects Dockerfile 5. Deploy + get URL: <https://eshara-api.railway.app>

- Test deployed API: GET /health, POST /predict/letter - Configure CORS to allow web + mobile app origins

LIST 6.3 — Web Deployment (Vercel)

TASK: [TO DO] Deploy Web Frontend to Vercel

Status: TO DO **Description:** - Platform: Vercel (free tier) - Steps: 1. Push web/ code to GitHub 2. Connect to Vercel, set root dir = web/ 3. Build command: `npm run build` 4. Output dir: `dist` 5. Set env var: `VITE_API_URL=https://eshara-api.railway.app` - Live URL: `https://eshara.vercel.app` - Test: end-to-end recognition from deployed web → deployed backend

LIST 6.4 — Mobile Deployment

TASK: [TO DO] Build Mobile App (Expo EAS)

Status: TO DO **Description:** - Platform: Expo EAS Build - Steps: 1. Install EAS CLI: `npm install -g eas-cli` 2. `eas build --platform android` → generates .apk 3. `eas build --platform ios` → generates .ipa 4. Distribute via Expo Go (development) or TestFlight / Google Play (production) - Configure `app.json`: name=ESHARA, version, bundle IDs

FOLDER 7: POLISH & OPTIONAL FEATURES

LIST 7.1 — Authentication & User History (Optional)

TASK: [TO DO] Supabase Integration (Auth + History)

Status: TO DO **Description:** - Platform: Supabase (free tier) - Tables: - **users:** id, email, preferred_language, created_at - **sessions:** id, user_id, transcript, language, duration, created_at - **corrections:** id, session_id, original, corrected, was_llm, created_at - Web: login/signup page + auth guards - Mobile: same auth flow - Backend: JWT validation middleware

LIST 7.2 — Performance & Accuracy Improvements

TASK: [TO DO] Benchmark & Optimize Letter Recognition

Status: TO DO **Description:** - Run accuracy benchmarks on ASL dataset: target >95% - Run accuracy benchmarks on ArSL dataset: target >93% - Optimize MediaPipe complexity setting (0 vs 1) for accuracy/speed tradeoff - Experiment with ensemble: MLP + MobileNetV2 → vote

TASK: [TO DO] Complete Arabic Word Model Training

Status: TO DO **Description:** - Acquire complete KArSL-502 dataset - Train ArSL word BiLSTM model to target accuracy - Save `arsl_word_lstm_model_best.h5` - Add to model registry and backend routes - Add to `shared_word_vocabulary.csv` Arabic entries

TASK: [TO DO] Expand Word Vocabulary Beyond 157 Words

Status: TO DO **Description:** - Current ASL word model: 157 word classes - Target: expand to 300+ words with more WLASL data - Use `ns1t_300.json` subset for next training iteration - Retrain, validate, update `asl_word_classes.csv` and `shared_word_vocabulary.csv`

LIST 7.3 — UX Improvements

TASK: [TO DO] Tutorial / Onboarding Mode

Status: TO DO **Description:** - Interactive tutorial showing each letter with expected hand shape - Sign chart reference page (A-Z ASL, Arabic alphabet ArSL) - Practice mode with guided feedback

TASK: [TO DO] Accessibility Features

Status: TO DO **Description:** - Text-to-speech output for recognized speech - High contrast mode - Text size adjustment - Screen reader compatibility

FOLDER 8: DOCUMENTATION & PROJECT MANAGEMENT

LIST 8.1 — Completed Documentation

TASK: [DONE] Full Deployment Guide (11 docs)

Status: DONE **Description:** Location: SLR Main/Deployment/docs/
Files completed: - 01_INVENTORY.md — what we have vs what we need (full audit) - 02_TECH_STACK.md — all languages, frameworks, tools - 03_ACCOUNTS_SETUP.md — services setup (GitHub, Railway, Vercel, Supabase, OpenAI) - 04_FOLDER_STRUCTURE.md — complete project structure - 05_BACKEND_GUIDE.md — FastAPI step-by-step (963 lines) - 06_WEB_FRONTEND_GUIDE.md — React step-by-step - 07_MOBILE_APP_GUIDE.md — React Native + Expo + TFLite - 08_MODEL_CONVERSION.md — .h5 → TFLite conversion - 09_DEPLOYMENT_CLOUD.md — Docker, Railway, Vercel - 10_TESTING_CHECKLIST.md — full test checklist - 11_TIMELINE.md — 3-week plan with daily tasks

TASK: [DONE] Architecture & Pipeline Docs

Status: DONE **Description:** - Words/Docs/MODEL_SUMMARY.md - Words/Docs/TEAM_QUICKSTART.md - Words/Docs/ARCHITECTURE_AND_PIPELINE.md - Words/Docs/LETTERS_WORDS_INTEGRATION.md - Words/Docs/DATASET_GUIDE.md - Letters/Guides/DEPLOYMENT_GUIDE.md - Letters/Guides/WORD_BUILDING_GUIDE.md - Letters/Guides/OPTIMIZATION_GUIDE.md

PRIORITY ORDER FOR NEXT STEPS

Recommended ClickUp Sprint Plan

Sprint 1 (Week 1) — Backend

Priority order: 1. Copy model files → backend/model_files/ (scripts/copy_models.py) 2. English Pipeline: model wrappers + inference coordinator + decoders 3. Arabic Pipeline: model wrappers + inference coordinator + decoders 4. Backend: config, main.py, model loader 5. Backend: letter_predictor, word_predictor, mode_detector 6. Backend: letter_decoder, word_decoder 7. Backend: REST routes (predict.py, health.py) 8. Backend: Pydantic schemas 9. Backend: WebSocket route 10. Basic tests with Swagger UI at localhost:8000

Sprint 2 (Week 2) — Web Frontend

Priority order: 1. Vite + React + TypeScript project setup 2. useMediaPipe hook 3. useWebSocket hook 4. Camera component 5. HandOverlay component 6. PredictionDisplay component 7. SentenceBuilder component 8. Recognition page (Recognize.tsx) 9. ModelIndicator + LanguageToggle 10. Home page + routing 11. i18n EN/AR translations 12. Polish + responsive design

Sprint 3 (Week 3) — Mobile + Deploy

Priority order: 1. Convert models to TFLite (scripts/convert_models.py + export_labels.py) 2. Expo project setup 3. TFLite model service 4. MediaPipe mobile service 5. Decoder service (TS port) 6. Recognition screen 7. Home + Settings screens 8. Docker build + test 9. Deploy backend to Railway 10. Deploy web to Vercel 11. Build mobile APK with EAS 12. End-to-end testing

Sprint 4 (Optional) — Polish

1. ArSL word model completion (blocker: KArSL-502 dataset)
 2. Supabase auth + history
 3. Tutorial/onboarding
 4. Performance benchmarks
 5. Vocabulary expansion to 300+ words
-

STATUS SUMMARY TABLE

Component	Sub-item	Status
ML Models	ASL Letter MLP	DONE
	ASL Letter MobileNetV2	DONE
	ArSL Letter MLP	DONE
	ArSL Letter MobileNetV2	DONE
	ASL Word BiLSTM + TemporalAttention	DONE
	ArSL Word BiLSTM	IN PROGRESS
Shared Infra	config/settings.py	DONE
	shared/models/model_loader.py	DONE
	shared/models/temporal_attention.py	DONE
	shared/utils/mediapipe_extractor.py	DONE
	shared/utils/mode_detector.py	DONE
	llm_agent/correction_agent.py	DONE
English Pipeline	llm_agent/prompts.py	DONE
	Model wrappers	TO DO
	Inference coordinator	TO DO
Arabic Pipeline	Letter/word decoders	TO DO
	Model wrappers	TO DO
	Inference coordinator	TO DO
Backend (FastAPI)	Letter/word decoders	TO DO
	Project setup	TO DO
	Config + main.py	TO DO
	Model loader	TO DO
	Letter predictor	TO DO
	Word predictor	TO DO
	Letter decoder	TO DO
	Word decoder	TO DO
	Mode detector	TO DO
	REST routes	TO DO
	WebSocket route	TO DO
	Pydantic schemas	TO DO
	Dockerfile	TO DO
	Railway config	TO DO
Web Frontend	Project setup	TO DO
	useMediaPipe hook	TO DO
	useWebSocket hook	TO DO
	Camera component	TO DO
	HandOverlay component	TO DO

Component	Sub-item	Status
Mobile App	PredictionDisplay component	TO DO
	SentenceBuilder component	TO DO
	ModeIndicator component	TO DO
	LanguageToggle component	TO DO
	Home page	TO DO
	Recognition page	TO DO
	API service	TO DO
	i18n translations	TO DO
	Vercel config	TO DO
	TFLite conversion	TO DO
	Expo project setup	TO DO
	TFLite model service	TO DO
	MediaPipe mobile service	TO DO
	Decoder service (TS)	TO DO
	Home screen	TO DO
	Recognition screen	TO DO
	Settings screen	TO DO
	CameraView component	TO DO
	ResultOverlay component	TO DO
	EAS build config	TO DO
Deployment	docker-compose.yml	TO DO
	Railway (backend)	TO DO
	Vercel (web)	TO DO
Documentation	Expo EAS (mobile)	TO DO
	All 11 deployment guides	DONE
	Architecture docs	DONE
Testing	README files	DONE
	Unit tests	TO DO
	Integration tests	TO DO
	End-to-end tests	TO DO